

Experiment ‘p_5_5_repeat’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 2

Verification attempts: 1

Problem Specification

Problem name: p_5_5_repeat

Natural language statement: null

Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- $|repeated_str| == n * |str|$
- forall $i :: 0 \leq i < n \implies repeated_str[i * |str| .. (i + 1) * |str|] == str$

Requires

- $n \geq 0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_5_repeat(str: string, n: int) returns (repeated_str: string)`

The method should respect the following contract:

requires `n >= 0`, ensures `|repeated_str| == n * |str|`, ensures forall `i :: 0 <= i < n ==>`
 `↪ repeated_str[i * |str| .. (i + 1) * |str| ] == str`

Produce and show only the Dafny body of this method, including the curly braces that
 `↪` surround it. Do not show the signature nor contract. You must not use 'assume' nor
 `↪` 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQvXl1fCIXpId7JZmYMPKsVkiRsQ

Model: gpt-5.5-2026-04-23

Created at: 1780673951

Finish reason: stop

Usage: 342 tokens in, and 9093 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
      ↪ (i + 1) * |str| <= |repeated_str|
    invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var prev := repeated_str;
    assert |prev| == k * |str|;
    assert forall i :: 0 <= i < k ==> prev[i * |str| .. (i + 1) * |str|] == str;

    repeated_str := prev + str;
    assert repeated_str == prev + str;
    assert |repeated_str| == |prev| + |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;
  }
}
```

```

assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↪ && (i + 1) * |str| <= |repeated_str| by {
  forall i | 0 <= i < k + 1
    ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
      ↪ repeated_str|
    {
      assert 0 <= |str|;
      assert 0 <= i;
      assert i <= i + 1;
      assert i + 1 <= k + 1;
      assert i * |str| <= (i + 1) * |str|;
      assert (i + 1) * |str| <= (k + 1) * |str|;
      assert (k + 1) * |str| == |repeated_str|;
    }
  }

forall i | 0 <= i < k + 1
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var a := i * |str|;
  var b := (i + 1) * |str|;
  assert 0 <= a;
  assert a <= b;
  assert b <= |repeated_str|;

  if i < k {
    assert i + 1 <= k;
    assert b <= k * |str|;
    assert b <= |prev|;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert repeated_str[a..b] == prev[a..b];
    assert prev[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert repeated_str[a..b] == repeated_str[|prev|..|prev| + |str|];
    assert repeated_str[|prev|..|prev| + |str|] == str;
  }
}

k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_5_5_repeat/program.dfy(11,14): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
11 | invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↳ && (i + 1) * |str| <= |repeated_str|
  | ~~~~~
```

```
problem_data/p_5_5_repeat/program.dfy(25,11): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
25 | assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
  ↳ && (i + 1) * |str| <= |repeated_str| by {
  | ~~~~~
```

```
problem_data/p_5_5_repeat/program.dfy(26,6): Warning: Could not find a trigger for this
  ↳ quantifier. Without a trigger, the quantifier may cause brittle verification. To
  ↳ silence this warning, add an explicit trigger using the {:trigger} attribute. For
  ↳ more information, see the section on quantifier instantiation rules in the
  ↳ reference manual.
```

```
|
26 | forall i | 0 <= i < k + 1
  | ~~~~~
```

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQydL1LgduJUOAY7x9hZkQUtwAwf

Model: gpt-5.5-2026-04-23

Created at: 1780674143

Finish reason: stop

Usage: 1967 tokens in, and 6463 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
```

```

invariant |repeated_str| == k * |str|
invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var prev := repeated_str;
  assert |prev| == k * |str|;

  forall i | 0 <= i < k
    ensures prev[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * |str|;
    var b := (i + 1) * |str|;
    assert 0 <= |str|;
    assert 0 <= i;
    assert i + 1 <= k;
    assert 0 <= a;
    assert a <= b;
    assert b <= k * |str|;
    assert b <= |prev|;
    assert prev == repeated_str;
    assert prev[a..b] == repeated_str[a..b];
    assert repeated_str[a..b] == str;
  }

  repeated_str := prev + str;
  assert repeated_str == prev + str;
  assert |repeated_str| == |prev| + |str|;
  assert |repeated_str| == (k + 1) * |str|;
  assert repeated_str[..|prev|] == prev;
  assert repeated_str[|prev|..] == str;

  forall i | 0 <= i < k + 1
    ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * |str|;
    var b := (i + 1) * |str|;
    assert 0 <= |str|;
    assert 0 <= i;
    assert i + 1 <= k + 1;
    assert 0 <= a;
    assert a <= b;
    assert b <= (k + 1) * |str|;
    assert b <= |repeated_str|;

    if i < k {
      assert i + 1 <= k;
      assert b <= k * |str|;
      assert b <= |prev|;
      assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
      assert repeated_str[a..b] == prev[a..b];
      assert prev[a..b] == str;
    } else {

```

```

    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == repeated_str[|prev|..|prev| + |str|];
    assert repeated_str[|prev|..|prev| + |str|] == repeated_str[|prev|..];
    assert repeated_str[|prev|..] == str;
  }
}

  k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_5_repeat/program.dfy(17,18): Error: upper bound below lower bound or
  ↪ above length of sequence

```

```

|
17 | ensures prev[i * |str| .. (i + 1) * |str|] == str
| ^

```

```

problem_data/p_5_5_repeat/program.dfy(26,6): Error: assertion might not hold

```

```

|
26 | assert b <= k * |str|;
| ~~~~~

```

```

problem_data/p_5_5_repeat/program.dfy(41,26): Error: lower bound out of range

```

```

|
41 | ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
| ^

```

Dafny program verifier finished with 1 verified, 3 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR2W39vz7L6uobRh2yNj6cQsHTdq

Model: gpt-5.5-2026-04-23

Created at: 1780674384

Finish reason: stop

Usage: 2763 tokens in, and 6198 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
      ↪ (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
      ↪ |] == str
  {
    var prev := repeated_str;

    assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i
      ↪ + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
      forall i | 0 <= i < k
        ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |prev
          ↪ | && prev[i * |str| .. (i + 1) * |str|] == str
      {
        var a := i * |str|;
        var b := (i + 1) * |str|;
        assert prev == repeated_str;
        assert 0 <= a && a <= b && b <= |repeated_str| && repeated_str[a..b] == str;
        assert |prev| == |repeated_str|;
        assert b <= |prev|;
        assert prev[a..b] == repeated_str[a..b];
      }
    }

    repeated_str := prev + str;
    assert repeated_str == prev + str;
    assert |repeated_str| == |prev| + |str|;
    assert |prev| == k * |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;

    assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
      ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |
      ↪ str|] == str by {
      forall i | 0 <= i < k + 1
        ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
          ↪ repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|] == str
      {
        var a := i * |str|;
        var b := (i + 1) * |str|;
        if i < k {
          assert 0 <= a && a <= b && b <= |prev| && prev[a..b] == str;
          assert |str| >= 0;
          assert |prev| <= |repeated_str|;
        }
      }
    }
  }
}
```

```

    assert b <= |repeated_str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert (repeated_str[..|prev|])[a..b] == prev[a..b];
    assert repeated_str[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert 0 <= a;
    assert a <= b;
    assert repeated_str[a..b] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[|prev|..|repeated_str|] == repeated_str[|prev|..];
    assert repeated_str[a..b] == str;
  }
}
}

k := k + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_5_5_repeat/program.dfy(11,14): Warning: Could not find a trigger for this
    → quantifier. Without a trigger, the quantifier may cause brittle verification. To
    → silence this warning, add an explicit trigger using the {:trigger} attribute. For
    → more information, see the section on quantifier instantiation rules in the
    → reference manual.
|
11 | invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
    → && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|]
    → == str
| ~~~~~~

problem_data/p_5_5_repeat/program.dfy(15,11): Warning: Could not find a trigger for this
    → quantifier. Without a trigger, the quantifier may cause brittle verification. To
    → silence this warning, add an explicit trigger using the {:trigger} attribute. For
    → more information, see the section on quantifier instantiation rules in the
    → reference manual.
|
15 | assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
    → (i + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
| ~~~~~~

```



```

problem_data/p_5_5_repeat/program.dfy(16,6): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
16 | forall i | 0 <= i < k
|   ~~~~~~

problem_data/p_5_5_repeat/program.dfy(37,11): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
37 | assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
    ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
    ↪ |] == str by {
|   ~~~~~~

problem_data/p_5_5_repeat/program.dfy(38,6): Warning: Could not find a trigger for this
    ↪ quantifier. Without a trigger, the quantifier may cause brittle verification. To
    ↪ silence this warning, add an explicit trigger using the {:trigger} attribute. For
    ↪ more information, see the section on quantifier instantiation rules in the
    ↪ reference manual.
|
38 | forall i | 0 <= i < k + 1
|   ~~~~~~

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR4trcvwCFWyRS411bpZIa7bXPWB

Model: gpt-5.5-2026-04-23

Created at: 1780674531

Finish reason: length

Usage: 5074 tokens in, and 3174 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
```

```

    requires n >= 0
    ensures |repeated_str| == n * |str|
    ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
        ↪ str
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| &&
        ↪ (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|
        ↪ ] == str
    {
      var prev := repeated_str;

      assert forall i :: 0 <= i < k ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i
        ↪ + 1) * |str| <= |prev| && prev[i * |str| .. (i + 1) * |str|] == str by {
        forall i | 0 <= i < k
          ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |prev|
            ↪ | && prev[i * |str| .. (i + 1) * |str|] == str
          {
            var a := i * |str|;
            var b := (i + 1) * |str|;
            assert prev == repeated_str;
            assert 0 <= a && a <= b && b <= |repeated_str| && repeated_str[a..b] == str;
            assert |prev| == |repeated_str|;
            assert b <= |prev|;
            assert prev[a..b] == repeated_str[a..b];
          }
        }
      }

      repeated_str := prev + str;
      assert repeated_str == prev + str;
      assert |repeated_str| == |prev| + |str|;
      assert |prev| == k * |str|;
      assert |repeated_str| == (k + 1) * |str|;
      assert repeated_str[..|prev|] == prev;
      assert repeated_str[|prev|..] == str;

      assert forall i :: 0 <= i < k + 1 ==> 0 <= i * |str| && i * |str| <= (i + 1) * |str|
        ↪ && (i + 1) * |str| <= |repeated_str| && repeated_str[i * |str| .. (i + 1) * |
        ↪ str|] == str by {
        forall i | 0 <= i < k + 1
          ensures 0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |
            ↪ repeated_str| && repeated_str[i * |str| .. (i + 1) * |str|] == str
          {
            var a := i * |str|;
            var b := (i + 1) * |str|;
            if i < k {
              assert 0 <= a && a <= b && b <= |prev| && prev[a..b] == str;

```

```

    assert |str| >= 0;
    assert |prev| <= |repeated_str|;
    assert b <= |repeated_str|;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[a..b] == (repeated_str[..|prev|])[a..b];
    assert (repeated_str[..|prev|])[a..b] == prev[a..b];
    assert repeated_str[a..b] == str;
  } else {
    assert i == k;
    assert a == k * |str|;
    assert a == |prev|;
    assert b == (k + 1) * |str|;
    assert b == |prev| + |str|;
    assert b == |repeated_str|;
    assert 0 <= a;
    assert a <= b;
    assert repeated_str[a..b] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[|prev|..|repeated_str|] == repeated_str[|prev|..];
    assert repeated_str[a..b] == str;
  }
}
}

  k := k + 1;
}
assert k == n;
}

```

Total Token Usage

Input tokens: 10146

Output tokens: 24928

Reasoning tokens: 22074

Sum of ‘total tokens’: 35074

Experiment Timings

Overall Experiment started at 1780673950879, ended at 1780674624892, lasting 674013ms (674.01 seconds)

Iteration #4 started at 1780674530861, ended at 1780674624876, lasting 94015ms (94.02 seconds)

Iteration #1 started at 1780673950881, ended at 1780674142558, lasting 191677ms (191.68 seconds)

Iteration #2 started at 1780674142561, ended at 1780674379681, lasting 237120ms (237.12 seconds)

Iteration #3 started at 1780674379691, ended at 1780674530861, lasting 151170ms (151.17 seconds)

Experiment 'p_5_9_f_cone_surface' Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_5_9_f_cone_surface

Natural language statement: null

Method signature: p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)

Ensures

- $-0.01 < (((\text{surface} / (3.14159 * r)) - r) * ((\text{surface} / (3.14159 * r)) - r)) - ((h * h) + (r * r)) < 0.01$

Requires

- $r > 0.0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)`

The method should respect the following contract:

requires $r > 0.0$, ensures $-0.01 < (((\text{surface} / (3.14159 * r)) - r) * ((\text{surface} / (3.14159 * r)) - r)) - ((h * h) + (r * r)) < 0.01$

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRBpi7gb2OvEuX5vNALyr4mg5DOd

Model: gpt-5.5-2026-04-23

Created at: 1780674961

Finish reason: stop

Usage: 362 tokens in, and 4644 tokens out

```
{
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert h * h + r * r >= 0.0;

  var x: real :| x * x == h * h + r * r;
  var c := 3.14159 * r;
  assert c > 0.0;
  assert c != 0.0;

  surface := c * (r + x);

  assert c == 3.14159 * r;
  assert surface / c == r + x;
  assert surface / (3.14159 * r) == r + x;
  assert (surface / (3.14159 * r)) - r == x;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == 0.0;
  assert -0.01 < 0.0;
  assert 0.0 < 0.01;
```

```
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_5_9_f_cone_surface/program.dfy(9,14): Error: cannot establish the
  ↳ existence of LHS values that satisfy the such-that predicate. Note, no trigger was
  ↳ found for the such-that predicate, which may be the reason the proof failed. To
  ↳ give a trigger explicitly, use the {:trigger} attribute. For more information, see
  ↳ the section on quantifier instantiation rules in the reference manual.
```

```
|
9 | var x: real :| x * x == h * h + r * r;
| ~~~~~
```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRE0RHbZADyvRXp9t6lgTRnmDoUY

Model: gpt-5.5-2026-04-23

Created at: 1780675096

Finish reason: stop

Usage: 872 tokens in, and 7799 tokens out

```
{
  var S := h * h + r * r;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert S >= 0.0;

  var lo := 0.0;
  var hi := S + 1.0;

  assert lo * lo <= S;
  assert S * S >= 0.0;
  calc {
    hi * hi - S;
    ==
    (S + 1.0) * (S + 1.0) - S;
    ==
    S * S + S + 1.0;
  }
  assert hi * hi - S >= 0.0;
  assert S <= hi * hi;
```

```

while 0.01 <= (hi - lo) * (hi + lo)
  invariant 0.0 <= lo <= hi
  invariant lo * lo <= S <= hi * hi
  decreases hi - lo
{
  var oldLo := lo;
  var oldHi := hi;
  var width := oldHi - oldLo;

  assert width >= 0.0;
  assert oldHi + oldLo >= 0.0;
  if width == 0.0 {
    assert (hi - lo) * (hi + lo) == 0.0;
    assert false;
  }
  assert width > 0.0;

  var mid := (oldLo + oldHi) / 2.0;
  assert mid - oldLo == width / 2.0;
  assert oldHi - mid == width / 2.0;
  assert width / 2.0 > 0.0;
  assert oldLo < mid < oldHi;

  if mid * mid <= S {
    lo := mid;
    assert hi == oldHi;
    assert lo * lo <= S;
    assert S <= hi * hi;
    assert hi - lo == width / 2.0;
    assert hi - lo < width;
    assert hi - lo < oldHi - oldLo;
  } else {
    hi := mid;
    assert lo == oldLo;
    assert S <= hi * hi;
    assert lo * lo <= S;
    assert hi - lo == width / 2.0;
    assert hi - lo < width;
    assert hi - lo < oldHi - oldLo;
  }
}

assert (hi - lo) * (hi + lo) < 0.01;
calc {
  hi * hi - lo * lo;
  ==
  (hi - lo) * (hi + lo);
}
assert S - lo * lo <= hi * hi - lo * lo;
assert S - lo * lo < 0.01;

```



```

assert lo * lo - S <= 0.0;
assert -0.01 < lo * lo - S;
assert lo * lo - S < 0.01;

var den := 3.14159 * r;
assert den > 0.0;
assert den != 0.0;

surface := den * (r + lo);

assert den == 3.14159 * r;
assert surface / den == r + lo;
assert surface / (3.14159 * r) == r + lo;
assert (surface / (3.14159 * r)) - r == lo;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == lo * lo - S;
assert -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((
    ↪ h * h) + (r * r)) < 0.01;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_9_f_cone_surface/program.dfy(25,2): Error: decreases expression might
    ↪ not decrease
|
25 | while 0.01 <= (hi - lo) * (hi + lo)
    | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRHisdfMwnmGFRCaFyIhsH8y7tmA

Model: gpt-5.5-2026-04-23

Created at: 1780675326

Finish reason: stop

Usage: 1866 tokens in, and 7970 tokens out

```

{
  var S := h * h + r * r;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert S >= 0.0;
}

```

```

var lo := 0.0;
var hi := S + 1.0;

assert lo * lo <= S;
assert S * S >= 0.0;
calc {
  hi * hi - S;
  ==
  (S + 1.0) * (S + 1.0) - S;
  ==
  S * S + S + 1.0;
}
assert hi * hi - S >= 0.0;
assert S <= hi * hi;

while 0.01 <= (hi - lo) * (hi + lo)
  invariant 0.0 <= lo <= hi
  invariant lo * lo <= S <= hi * hi
  decreases hi - lo
{
  var oldLo := lo;
  var oldHi := hi;
  var width := oldHi - oldLo;

  assert width >= 0.0;
  assert oldHi + oldLo >= 0.0;
  if width == 0.0 {
    assert (hi - lo) * (hi + lo) == 0.0;
    assert false;
  }
  assert width > 0.0;

  var mid := (oldLo + oldHi) / 2.0;
  assert mid - oldLo == width / 2.0;
  assert oldHi - mid == width / 2.0;
  assert width / 2.0 > 0.0;
  assert oldLo < mid < oldHi;

  if mid * mid <= S {
    lo := mid;
    assert hi == oldHi;
    assert lo * lo <= S;
    assert S <= hi * hi;
    assert hi - lo == width / 2.0;
  } else {
    hi := mid;
    assert lo == oldLo;
    assert S <= hi * hi;
    assert lo * lo <= S;
    assert hi - lo == width / 2.0;
  }
}

```

```

    }

    assert hi - lo == width / 2.0;
    assert width / 2.0 < width;
    assert hi - lo < width;
    assert width == oldHi - oldLo;
    assert hi - lo < oldHi - oldLo;
  }

  assert (hi - lo) * (hi + lo) < 0.01;
  calc {
    hi * hi - lo * lo;
    ==
    (hi - lo) * (hi + lo);
  }
  assert S - lo * lo <= hi * hi - lo * lo;
  assert S - lo * lo < 0.01;
  assert lo * lo - S <= 0.0;
  assert -0.01 < lo * lo - S;
  assert lo * lo - S < 0.01;

  var den := 3.14159 * r;
  assert den > 0.0;
  assert den != 0.0;

  surface := den * (r + lo);

  assert den == 3.14159 * r;
  assert surface / den == r + lo;
  assert surface / (3.14159 * r) == r + lo;
  assert (surface / (3.14159 * r)) - r == lo;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == lo * lo - S;
  assert -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((
    ↪ h * h) + (r * r)) < 0.01;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_9_f_cone_surface/program.dfy(25,2): Error: decreases expression might
    ↪ not decrease
|
25 | while 0.01 <= (hi - lo) * (hi + lo)
|   ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRLqsgkxar9sxxft6xNeMmYH7S

Model: gpt-5.5-2026-04-23

Created at: 1780675582

Finish reason: length

Usage: 2941 tokens in, and 6487 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)
  requires r > 0.0
  ensures -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)
    → ) - ((h * h) + (r * r)) < 0.01
{
  var S := h * h + r * r;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert S >= 0.0;

  var lo := 0.0;
  var hi := S + 1.0;

  assert lo * lo <= S;
  assert S * S >= 0.0;
  calc {
    hi * hi - S;
    ==
    (S + 1.0) * (S + 1.0) - S;
    ==
    S * S + S + 1.0;
  }
  assert hi * hi - S >= 0.0;
  assert S <= hi * hi;

  while 0.01 <= (hi - lo) * (hi + lo)
    invariant 0.0 <= lo <= hi
    invariant lo * lo <= S <= hi * hi
    decreases hi - lo
  {
    var oldLo := lo;
    var oldHi := hi;
```

```

var width := oldHi - oldLo;

assert width >= 0.0;
assert oldHi + oldLo >= 0.0;
if width == 0.0 {
  assert (hi - lo) * (hi + lo) == 0.0;
  assert false;
}
assert width > 0.0;

var mid := (oldLo + oldHi) / 2.0;
assert mid - oldLo == width / 2.0;
assert oldHi - mid == width / 2.0;
assert width / 2.0 > 0.0;
assert oldLo < mid < oldHi;

if mid * mid <= S {
  lo := mid;
  assert hi == oldHi;
  assert lo * lo <= S;
  assert S <= hi * hi;
  assert hi - lo == width / 2.0;
} else {
  hi := mid;
  assert lo == oldLo;
  assert S <= hi * hi;
  assert lo * lo <= S;
  assert hi - lo == width / 2.0;
}

assert hi - lo == width / 2.0;
assert width / 2.0 < width;
assert hi - lo < width;
assert width == oldHi - oldLo;
assert hi - lo < oldHi - oldLo;
}

assert (hi - lo) * (hi + lo) < 0.01;
calc {
  hi * hi - lo * lo;
  ==
  (hi - lo) * (hi + lo);
}
assert S - lo * lo <= hi * hi - lo * lo;
assert S - lo * lo < 0.01;
assert lo * lo - S <= 0.0;
assert -0.01 < lo * lo - S;
assert lo * lo - S < 0.01;

var den := 3.14159 * r;
assert den > 0.0;

```

```

assert den != 0.0;

surface := den * (r + lo);

assert den == 3.14159 * r;
assert surface / den == r + lo;
assert surface / (3.14159 * r) == r + lo;
assert (surface / (3.14159 * r)) - r == lo;
assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
    ↪ (r * r)) == lo * lo - S;
assert -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((
    ↪ h * h) + (r * r)) < 0.01;
}

```

Total Token Usage

Input tokens: 6041

Output tokens: 26900

Reasoning tokens: 24576

Sum of 'total tokens': 32941

Experiment Timings

Overall Experiment started at 1780674961074, ended at 1780675742234, lasting 781160ms (781.16 seconds)

Iteration #4 started at 1780675581546, ended at 1780675742234, lasting 160688ms (160.69 seconds)

Iteration #1 started at 1780674961074, ended at 1780675096472, lasting 135398ms (135.40 seconds)

Iteration #2 started at 1780675096477, ended at 1780675322291, lasting 225814ms (225.81 seconds)

Iteration #3 started at 1780675322291, ended at 1780675581546, lasting 259255ms (259.26 seconds)

Experiment 'p_5_1_a_smallest' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_1_a_smallest

Natural language statement: null

Method signature: p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)

Ensures

- `smallest <= x && smallest <= y && smallest <= z`
- `smallest == x || smallest == y || smallest == z`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)`

The method should respect the following contract:

```
ensures smallest <= x && smallest <= y && smallest <= z, ensures smallest == x ||  
    ↪ smallest == y || smallest == z
```

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQr6x9mWA9RPtAfyFkNEpyQhCWlg

Model: gpt-5.5-2026-04-23

Created at: 1780673676

Finish reason: stop

Usage: 302 tokens in, and 386 tokens out

```
{  
  if x <= y {  
    if x <= z {  
      smallest := x;  
    } else {  
      smallest := z;  
    }  
  } else {  
    if y <= z {  
      smallest := y;  
    } else {  
      smallest := z;  
    }  
  }  
}
```

Final Program

```
method p_5_1_a_smallest(x: real, y: real, z: real) returns (smallest: real)
  ensures smallest <= x && smallest <= y && smallest <= z
  ensures smallest == x || smallest == y || smallest == z
{
  if x <= y {
    if x <= z {
      smallest := x;
    } else {
      smallest := z;
    }
  } else {
    if y <= z {
      smallest := y;
    } else {
      smallest := z;
    }
  }
}
```

Total Token Usage

Input tokens: 302

Output tokens: 386

Reasoning tokens: 297

Sum of ‘total tokens’: 688

Experiment Timings

Overall Experiment started at 1780673673828, ended at 1780673724370, lasting 50542ms (50.54 seconds)

Iteration #1 started at 1780673673836, ended at 1780673724369, lasting 50533ms (50.53 seconds)

Experiment 'p_5_1_b_average' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_1_b_average

Natural language statement: null

Method signature: p_5_1_b_average(x: real, y: real, z: real) returns (average: real)

Ensures

- $\text{average} == (x + y + z) / 3.0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_1_b_average(x: real, y: real, z: real) returns (average: real)`

The method should respect the following contract:

`ensures average == (x + y + z) / 3.0`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQsBmt5D7bujIrmfxBznFUTUYYuf

Model: gpt-5.5-2026-04-23

Created at: 1780673743

Finish reason: stop

Usage: 278 tokens in, and 108 tokens out

```
{  
  average := (x + y + z) / 3.0;  
}
```

Final Program

```
method p_5_1_b_average(x: real, y: real, z: real) returns (average: real)  
  ensures average == (x + y + z) / 3.0  
{  
  average := (x + y + z) / 3.0;  
}
```

Total Token Usage

Input tokens: 278

Output tokens: 108

Reasoning tokens: 71

Sum of ‘total tokens’: 386

Experiment Timings

Overall Experiment started at 1780673743271, ended at 1780673760334, lasting 17063ms (17.06 seconds)

Iteration #1 started at 1780673743285, ended at 1780673760334, lasting 17049ms (17.05 seconds)

Experiment 'p_5_2_a_all_the_same' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_a_all_the_same

Natural language statement: null

Method signature: p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)

Ensures

- all_same == (x == y && y == z)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)`

The method should respect the following contract:

`ensures all_same == (x == y && y == z)`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQsUGLwaAyzvFj4XjL8HqyH4RdUO

Model: gpt-5.5-2026-04-23

Created at: 1780673762

Finish reason: stop

Usage: 278 tokens in, and 115 tokens out

```
{
  all_same := x == y && y == z;
}
```

Final Program

```
method p_5_2_a_all_the_same(x: real, y: real, z: real) returns (all_same: bool)
  ensures all_same == (x == y && y == z)
{
  all_same := x == y && y == z;
}
```

Total Token Usage

Input tokens: 278

Output tokens: 115

Reasoning tokens: 82

Sum of ‘total tokens’: 393

Experiment Timings

Overall Experiment started at 1780673762024, ended at 1780673794947, lasting 32923ms (32.92 seconds)

Iteration #1 started at 1780673762024, ended at 1780673794946, lasting 32922ms (32.92 seconds)

Experiment 'p_5_2_b_all_different' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_b_all_different

Natural language statement: null

Method signature: p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)

Ensures

- all_diff == (x != y && y != z && x != z)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)`

The method should respect the following contract:

`ensures all_diff == (x != y && y != z && x != z)`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQt3ZGMLJJ4qFaizMvTKaS3X7piB

Model: gpt-5.5-2026-04-23

Created at: 1780673797

Finish reason: stop

Usage: 286 tokens in, and 94 tokens out

```
{  
  all_diff := x != y && y != z && x != z;  
}
```

Final Program

```
method p_5_2_b_all_different(x: real, y: real, z: real) returns (all_diff: bool)  
  ensures all_diff == (x != y && y != z && x != z)  
{  
  all_diff := x != y && y != z && x != z;  
}
```

Total Token Usage

Input tokens: 286

Output tokens: 94

Reasoning tokens: 57

Sum of ‘total tokens’: 380

Experiment Timings

Overall Experiment started at 1780673796434, ended at 1780673817214, lasting 20780ms (20.78 seconds)

Iteration #1 started at 1780673796436, ended at 1780673817214, lasting 20778ms (20.78 seconds)

Experiment ‘p_5_2_c_sorted’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_2_c_sorted

Natural language statement: null

Method signature: p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)

Ensures

- `is_sorted == (x <= y && y <= z)`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)`

The method should respect the following contract:

`ensures is_sorted == (x <= y && y <= z)`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQtSBSppboKwbcLLKC4WKyH8jG2L

Model: gpt-5.5-2026-04-23

Created at: 1780673822

Finish reason: stop

Usage: 274 tokens in, and 117 tokens out

```
{  
  is_sorted := x <= y && y <= z;  
}
```

Final Program

```
method p_5_2_c_sorted(x: real, y: real, z: real) returns (is_sorted: bool)  
  ensures is_sorted == (x <= y && y <= z)  
{  
  is_sorted := x <= y && y <= z;  
}
```

Total Token Usage

Input tokens: 274

Output tokens: 117

Reasoning tokens: 84

Sum of ‘total tokens’: 391

Experiment Timings

Overall Experiment started at 1780673821546, ended at 1780673860487, lasting 38941ms (38.94 seconds)

Iteration #1 started at 1780673821563, ended at 1780673860487, lasting 38924ms (38.92 seconds)

Experiment 'p_5_3_first_digit' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_3_first_digit

Natural language statement: null

Method signature: p_5_3_first_digit(n: int) returns (first: int)

Ensures

- `0 <= first < 10`
- `intToSeq(n)[0] == first`

Requires

- `0 <= n`

Functional Code Given

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_3_first_digit(n: int)` returns `(first: int)`

The method should respect the following contract:

`requires 0 <= n, ensures 0 <= first < 10, ensures intToSeq(n)[0] == first`

The contract uses the following dafny code:

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQu9r22AVSoYJLhBxNDD75tf4fyH

Model: gpt-5.5-2026-04-23

Created at: 1780673865

Finish reason: stop

Usage: 464 tokens in, and 570 tokens out

```
{
  if n < 10 {
    first := n;
  } else {
    first := p_5_3_first_digit(n / 10);
  }
}
```

Final Program

```
function intToSeq(n: int): seq<int>
  requires n >= 0
  ensures forall d :: d in intToSeq(n) ==> 0 <= d < 10
{
  if n < 10 then [n]
  else intToSeq(n / 10) + [n % 10]
}

method p_5_3_first_digit(n: int) returns (first: int)
  requires 0 <= n
  ensures 0 <= first < 10
  ensures intToSeq(n)[0] == first
{
  if n < 10 {
    first := n;
  } else {
    first := p_5_3_first_digit(n / 10);
  }
}
```

Total Token Usage

Input tokens: 464

Output tokens: 570

Reasoning tokens: 512

Sum of 'total tokens': 1034

Experiment Timings

Overall Experiment started at 1780673864366, ended at 1780673904559, lasting 40193ms (40.19 seconds)

Iteration #1 started at 1780673864382, ended at 1780673904556, lasting 40174ms (40.17 seconds)

Experiment 'p_5_4_middle' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_4_middle

Natural language statement: null

Method signature: p_5_4_middle(str: string) returns (middle_str: string)

Ensures

- if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
- if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str == str[|str| / 2 - 1 .. (|str| / 2) + 1]

Requires

- 0 < |str|

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_4_middle(str: string) returns (middle_str: string)`

The method should respect the following contract:

requires $0 < |str|$, ensures if $|str| \% 2 == 1$ then $|middle_str| == 1$ else $|middle_str| == 2$, ensures if $|str| \% 2 == 1$ then $middle_str[0] == str[|str| / 2]$ else $middle_str == str[|str| / 2 - 1 .. (|str| / 2) + 1]$

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQuo7C7XhA8jeyp7UHL5wF5aRQAM

Model: gpt-5.5-2026-04-23

Created at: 1780673906

Finish reason: stop

Usage: 410 tokens in, and 674 tokens out

```
{
  var mid := |str| / 2;
  if |str| % 2 == 1 {
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert |str| % 2 == 0;
    assert 2 <= |str|;
    assert 1 <= mid;
    assert mid + 1 <= |str|;
    middle_str := str[mid - 1 .. mid + 1];
    assert |middle_str| == 2;
  }
}
```

Final Program

```

method p_5_4_middle(str: string) returns (middle_str: string)
  requires 0 < |str|
  ensures if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
  ensures if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str ==
    ↪ str[|str| / 2 - 1 .. (|str| / 2) + 1]
{
  var mid := |str| / 2;
  if |str| % 2 == 1 {
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert |str| % 2 == 0;
    assert 2 <= |str|;
    assert 1 <= mid;
    assert mid + 1 <= |str|;
    middle_str := str[mid - 1 .. mid + 1];
    assert |middle_str| == 2;
  }
}

```

Total Token Usage

Input tokens: 410

Output tokens: 674

Reasoning tokens: 512

Sum of 'total tokens': 1084

Experiment Timings

Overall Experiment started at 1780673906254, ended at 1780673949102, lasting 42848ms (42.85 seconds)

Iteration #1 started at 1780673906261, ended at 1780673949102, lasting 42841ms (42.84 seconds)

Experiment ‘p_5_7_count_words’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_7_count_words

Natural language statement: null

Method signature: p_5_7_count_words(str: string) returns (word_count: int)

Ensures

- word_count == count_words(str)

Functional Code Given

```
function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
  var s2 := remove_leading_spaces(s);
```

```
    if |s2| > 0 then 1 + count_words(remove_word(s2))  
    else 0  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_7_count_words(str: string)` returns `(word_count: int)`

The method should respect the following contract:

`ensures word_count == count_words(str)`

The contract uses the following dafny code:

```
function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
  var s2 := remove_leading_spaces(s);
  if |s2| > 0 then 1 + count_words(remove_word(s2))
  else 0
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR6SpMhRsU1gzQmrWrNi2P7wLFcH

Model: gpt-5.5-2026-04-23

Created at: 1780674628

Finish reason: stop

Usage: 692 tokens in, and 4941 tokens out

```
{
  word_count := 0;
  var i := 0;
  assert str[0..] == str;

  while i < |str|
    invariant 0 <= i <= |str|
    invariant 0 <= word_count
    invariant word_count + count_words(str[i..]) == count_words(str)
  {
    if str[i] == ' ' {
      assert 0 <= i < |str|;
      assert |str[i..]| > 0;
      assert (str[i..])[0] == ' ';
      assert (str[i..])[1..] == str[i + 1..];
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces((str[i..])[1..]);
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces(str[i + 1..]);
      assert count_words(str[i..]) == count_words(str[i + 1..]);
      assert word_count + count_words(str[i + 1..]) == count_words(str);
      i := i + 1;
    } else {
      var start := i;
      var old_count := word_count;
      assert 0 <= start < |str|;
      assert str[start] != ' ';
      assert |str[start..]| > 0;
      assert (str[start..])[0] != ' ';
      assert remove_leading_spaces(str[start..]) == str[start..];
      assert count_words(str[start..]) == 1 + count_words(remove_word(str[start..]));
      assert old_count + count_words(str[start..]) == count_words(str);
      word_count := old_count + 1;
      assert word_count + count_words(remove_word(str[start..])) == count_words(str);
      assert i == start;
      assert word_count + count_words(remove_word(str[i..])) == count_words(str);

      while i < |str| && str[i] != ' '
        invariant start <= i <= |str|
        invariant 0 <= word_count
        invariant word_count + count_words(remove_word(str[i..])) == count_words(str)
      {
        assert 0 <= i < |str|;
        assert |str[i..]| > 0;
        assert (str[i..])[0] != ' ';
```

```

    assert (str[i..])[1..] == str[i + 1..];
    assert remove_word(str[i..]) == remove_word((str[i..])[1..]);
    assert remove_word(str[i..]) == remove_word(str[i + 1..]);
    assert word_count + count_words(remove_word(str[i + 1..])) == count_words(str);
    i := i + 1;
}

if i == |str| {
  assert str[i..] == "";
  assert remove_word(str[i..]) == str[i..];
} else {
  assert i < |str|;
  assert str[i] == ' ';
  assert |str[i..]| > 0;
  assert (str[i..])[0] == ' ';
  assert remove_word(str[i..]) == str[i..];
}
assert word_count + count_words(str[i..]) == count_words(str);
}
}

assert i == |str|;
assert str[i..] == "";
assert count_words(str[i..]) == 0;
assert word_count == count_words(str);
}

```

Final Program

```

function remove_leading_spaces(s: string): (result: string)
  ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] == ' ' then remove_leading_spaces(s[1..])
  else s
}

function remove_word(s: string): (result: string)
  ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
  if |s| == 0 then s
  else if s[0] != ' ' then remove_word(s[1..])
  else s
}

function count_words(s: string): int
  decreases |s|
{
  var s2 := remove_leading_spaces(s);
  if |s2| > 0 then 1 + count_words(remove_word(s2))
}

```



```

    else 0
}

method p_5_7_count_words(str: string) returns (word_count: int)
    ensures word_count == count_words(str)
{
    word_count := 0;
    var i := 0;
    assert str[0..] == str;

    while i < |str|
        invariant 0 <= i <= |str|
        invariant 0 <= word_count
        invariant word_count + count_words(str[i..]) == count_words(str)
    {
        if str[i] == ' ' {
            assert 0 <= i < |str|;
            assert |str[i..]| > 0;
            assert (str[i..])[0] == ' ';
            assert (str[i..])[1..] == str[i + 1..];
            assert remove_leading_spaces(str[i..]) == remove_leading_spaces((str[i..])[1..]);
            assert remove_leading_spaces(str[i..]) == remove_leading_spaces(str[i + 1..]);
            assert count_words(str[i..]) == count_words(str[i + 1..]);
            assert word_count + count_words(str[i + 1..]) == count_words(str);
            i := i + 1;
        } else {
            var start := i;
            var old_count := word_count;
            assert 0 <= start < |str|;
            assert str[start] != ' ';
            assert |str[start..]| > 0;
            assert (str[start..])[0] != ' ';
            assert remove_leading_spaces(str[start..]) == str[start..];
            assert count_words(str[start..]) == 1 + count_words(remove_word(str[start..]));
            assert old_count + count_words(str[start..]) == count_words(str);
            word_count := old_count + 1;
            assert word_count + count_words(remove_word(str[start..])) == count_words(str);
            assert i == start;
            assert word_count + count_words(remove_word(str[i..])) == count_words(str);

            while i < |str| && str[i] != ' '
                invariant start <= i <= |str|
                invariant 0 <= word_count
                invariant word_count + count_words(remove_word(str[i..])) == count_words(str)
            {
                assert 0 <= i < |str|;
                assert |str[i..]| > 0;
                assert (str[i..])[0] != ' ';
                assert (str[i..])[1..] == str[i + 1..];
                assert remove_word(str[i..]) == remove_word((str[i..])[1..]);
                assert remove_word(str[i..]) == remove_word(str[i + 1..]);
            }

```

```

    assert word_count + count_words(remove_word(str[i + 1..])) == count_words(str);
    i := i + 1;
}

if i == |str| {
    assert str[i..] == "";
    assert remove_word(str[i..]) == str[i..];
} else {
    assert i < |str|;
    assert str[i] == ' ';
    assert |str[i..]| > 0;
    assert (str[i..])[0] == ' ';
    assert remove_word(str[i..]) == str[i..];
}
assert word_count + count_words(str[i..]) == count_words(str);
}
}

assert i == |str|;
assert str[i..] == "";
assert count_words(str[i..]) == 0;
assert word_count == count_words(str);
}

```

Total Token Usage

Input tokens: 692

Output tokens: 4941

Reasoning tokens: 4096

Sum of 'total tokens': 5633

Experiment Timings

Overall Experiment started at 1780674628296, ended at 1780674760006, lasting 131710ms (131.71 seconds)

Iteration #1 started at 1780674628297, ended at 1780674760006, lasting 131709ms (131.71 seconds)

Experiment 'p_5_9_a_sphere_volume' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_a_sphere_volume

Natural language statement: null

Method signature: p_5_9_a_sphere_volume(r: real) returns (volume: real)

Ensures

- `volume == (4.0 / 3.0) * 3.14159 * r * r * r`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_a_sphere_volume(r: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == (4.0 / 3.0) * 3.14159 * r * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR8h4e2FxqWzBjOIgUq598h51EeU

Model: gpt-5.5-2026-04-23

Created at: 1780674767

Finish reason: stop

Usage: 286 tokens in, and 115 tokens out

```
{  
  volume := (4.0 / 3.0) * 3.14159 * r * r * r;  
}
```

Final Program

```
method p_5_9_a_sphere_volume(r: real) returns (volume: real)  
  ensures volume == (4.0 / 3.0) * 3.14159 * r * r * r  
{  
  volume := (4.0 / 3.0) * 3.14159 * r * r * r;  
}
```

Total Token Usage

Input tokens: 286

Output tokens: 115

Reasoning tokens: 68

Sum of ‘total tokens’: 401

Experiment Timings

Overall Experiment started at 1780674766447, ended at 1780674791212, lasting 24765ms (24.77 seconds)

Iteration #1 started at 1780674766451, ended at 1780674791212, lasting 24761ms (24.76 seconds)

Experiment 'p_5_9_b_sphere_surface' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_b_sphere_surface

Natural language statement: null

Method signature: p_5_9_b_sphere_surface(r: real) returns (surface: real)

Ensures

- `surface == 4.0 * 3.14159 * r * r`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_b_sphere_surface(r: real) returns (surface: real)`

The method should respect the following contract:

`ensures surface == 4.0 * 3.14159 * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR9AYBAGeqdKwliTJUplID40A39N

Model: gpt-5.5-2026-04-23

Created at: 1780674796

Finish reason: stop

Usage: 270 tokens in, and 144 tokens out

```
{  
  surface := 4.0 * 3.14159 * r * r;  
}
```

Final Program

```
method p_5_9_b_sphere_surface(r: real) returns (surface: real)  
  ensures surface == 4.0 * 3.14159 * r * r  
{  
  surface := 4.0 * 3.14159 * r * r;  
}
```

Total Token Usage

Input tokens: 270

Output tokens: 144

Reasoning tokens: 105

Sum of ‘total tokens’: 414

Experiment Timings

Overall Experiment started at 1780674796069, ended at 1780674829974, lasting 33905ms (33.91 seconds)

Iteration #1 started at 1780674796070, ended at 1780674829974, lasting 33904ms (33.90 seconds)

Experiment 'p_5_9_c_cylinder_volume' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_c_cylinder_volume

Natural language statement: null

Method signature: p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)

Ensures

- `volume == 3.14159 * r * r * h`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == 3.14159 * r * r * h`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnR9pr1X373rJaG2dNUVrNLibkcmX

Model: gpt-5.5-2026-04-23

Created at: 1780674837

Finish reason: stop

Usage: 272 tokens in, and 109 tokens out

```
{  
  volume := 3.14159 * r * r * h;  
}
```

Final Program

```
method p_5_9_c_cylinder_volume(r: real, h: real) returns (volume: real)  
  ensures volume == 3.14159 * r * r * h  
{  
  volume := 3.14159 * r * r * h;  
}
```

Total Token Usage

Input tokens: 272

Output tokens: 109

Reasoning tokens: 73

Sum of ‘total tokens’: 381

Experiment Timings

Overall Experiment started at 1780674833090, ended at 1780674870225, lasting 37135ms (37.14 seconds)

Iteration #1 started at 1780674833160, ended at 1780674870225, lasting 37065ms (37.07 seconds)

Experiment 'p_5_9_d_cylinder_surface' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_d_cylinder_surface

Natural language statement: null

Method signature: p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)

Ensures

- $\text{surface} == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)`

The method should respect the following contract:

`ensures surface == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRAYanjwUw2DemJdppFxxPBrOruo

Model: gpt-5.5-2026-04-23

Created at: 1780674882

Finish reason: stop

Usage: 308 tokens in, and 157 tokens out

```
{  
  surface := 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r;  
}
```

Final Program

```
method p_5_9_d_cylinder_surface(r: real, h: real) returns (surface: real)  
  ensures surface == 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r  
{  
  surface := 2.0 * 3.14159 * r * h + 2.0 * 3.14159 * r * r;  
}
```

Total Token Usage

Input tokens: 308

Output tokens: 157

Reasoning tokens: 103

Sum of ‘total tokens’: 465

Experiment Timings

Overall Experiment started at 1780674880702, ended at 1780674915322, lasting 34620ms (34.62 seconds)

Iteration #1 started at 1780674880706, ended at 1780674915322, lasting 34616ms (34.62 seconds)

Experiment 'p_5_9_e_cone_volume' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_9_e_cone_volume

Natural language statement: null

Method signature: p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)

Ensures

- `volume == (1.0 / 3.0) * 3.14159 * r * r * h`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)`

The method should respect the following contract:

`ensures volume == (1.0 / 3.0) * 3.14159 * r * r * h`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnRB7YCCFvPOy9eyhEermG8Pm4x8M

Model: gpt-5.5-2026-04-23

Created at: 1780674917

Finish reason: stop

Usage: 294 tokens in, and 133 tokens out

```
{  
  volume := (1.0 / 3.0) * 3.14159 * r * r * h;  
}
```

Final Program

```
method p_5_9_e_cone_volume(r: real, h: real) returns (volume: real)  
  ensures volume == (1.0 / 3.0) * 3.14159 * r * r * h  
{  
  volume := (1.0 / 3.0) * 3.14159 * r * r * h;  
}
```

Total Token Usage

Input tokens: 294

Output tokens: 133

Reasoning tokens: 86

Sum of ‘total tokens’: 427

Experiment Timings

Overall Experiment started at 1780674916476, ended at 1780674957728, lasting 41252ms (41.25 seconds)

Iteration #1 started at 1780674916476, ended at 1780674957728, lasting 41252ms (41.25 seconds)

